

An Efficient Certificate-Based Aggregate Signature Scheme With Provable Security for Industrial Internet of Things

Reproducing an attack, testing its repair, and auditing the cost claims

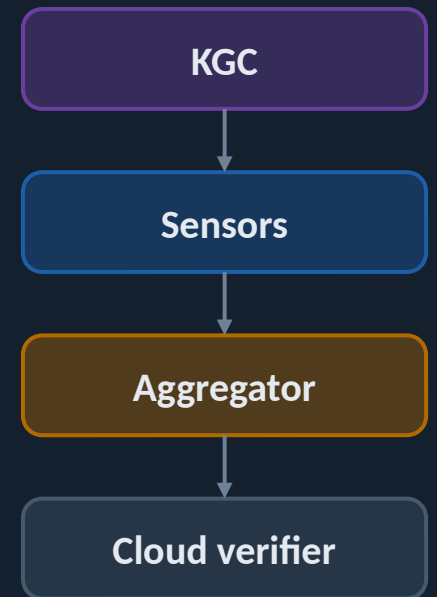
PAPER UNDER STUDY

Zirui Qiao et al. • IEEE Systems Journal, Vol. 17, No. 1, 2023

DOI: 10.1109/JSYST.2022.3188012

Anurag Kaushal (2025202013) • Pranav Vyas (2025202015) • Taufique (2025202007)

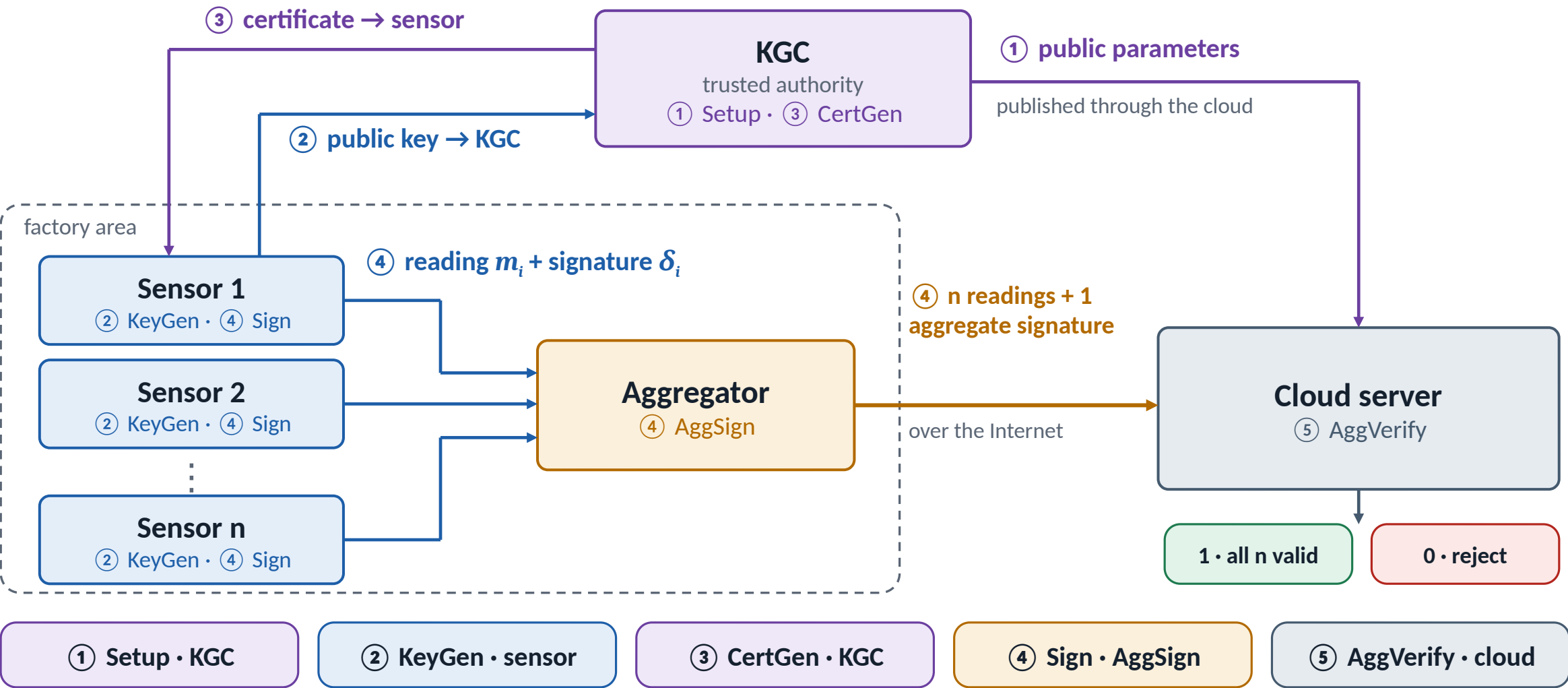
Research in Information Security • IIIT Hyderabad



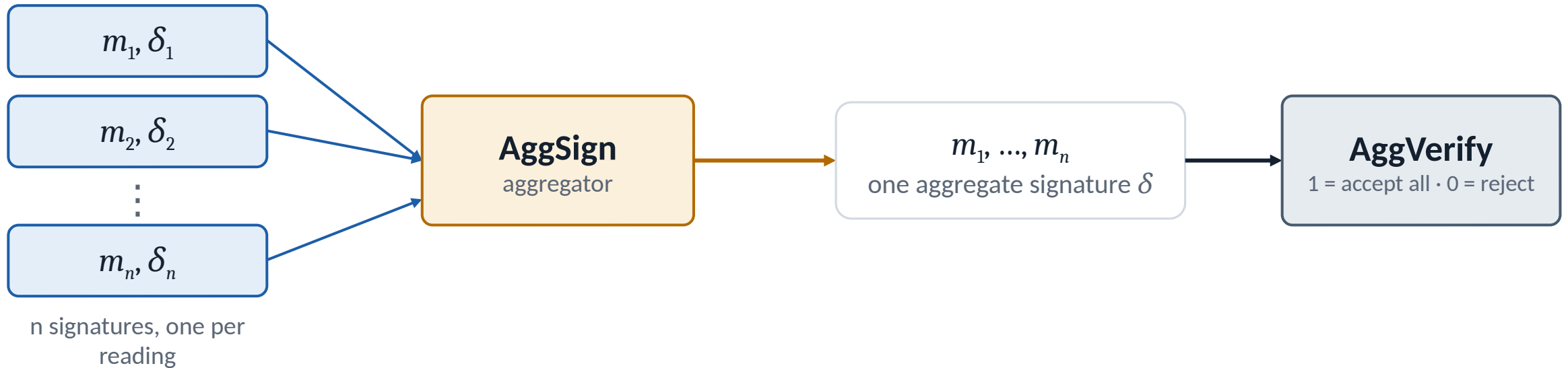
the four parties, with one colour each throughout

Can a malicious key authority forge sensor data?

Sensors sign, an aggregator combines, the cloud verifies



One verification decision instead of n



Without aggregation

n signatures sent • n verification calls

With aggregation

one aggregate signature • one verification call

Aggregation shrinks what is sent and gives one accept-or-reject decision for all n readings.

Who holds the signing key?

Public-key infrastructure

user's key

- A CA certifies the key
- Certificates must be stored, distributed and revoked

User alone signs

Identity-based

KGC computes the key

- The KGC derives the whole private key
- The KGC can sign as anyone: key escrow

KGC alone can sign

Certificate-based

sk_i • user

$Cert_i$ • KGC

- The user makes sk_i and pk_i itself
- The KGC issues $Cert_i$; signing needs both

Neither alone can sign

Design goal: the KGC alone must not be able to sign.

What if the KGC is the attacker?

THE THREAT

A KGC that goes bad holds every certificate.

- Postbank, South Africa: an employee took the data centre's master secret key and stole 3.2 million dollars (cited in the paper, §I-C).
- The paper's requirement: a practical CBAS scheme must stay secure even if the KGC is malicious.

Q1

Can a malicious KGC forge signatures in Verma et al.'s scheme?

Paper: yes (§IV). We run the attack.

Q2

Does Qiao et al.'s repair stop it, and which change is responsible?

Paper: yes (§V). We test it, and remove one change.

Q3

Is the repaired scheme really as cheap as Verma's?

Paper: identical cost. We recount and measure.

PAPER

claim by Qiao et al.

REPRODUCED

we re-ran it

IMPLEMENTATION

our design choice

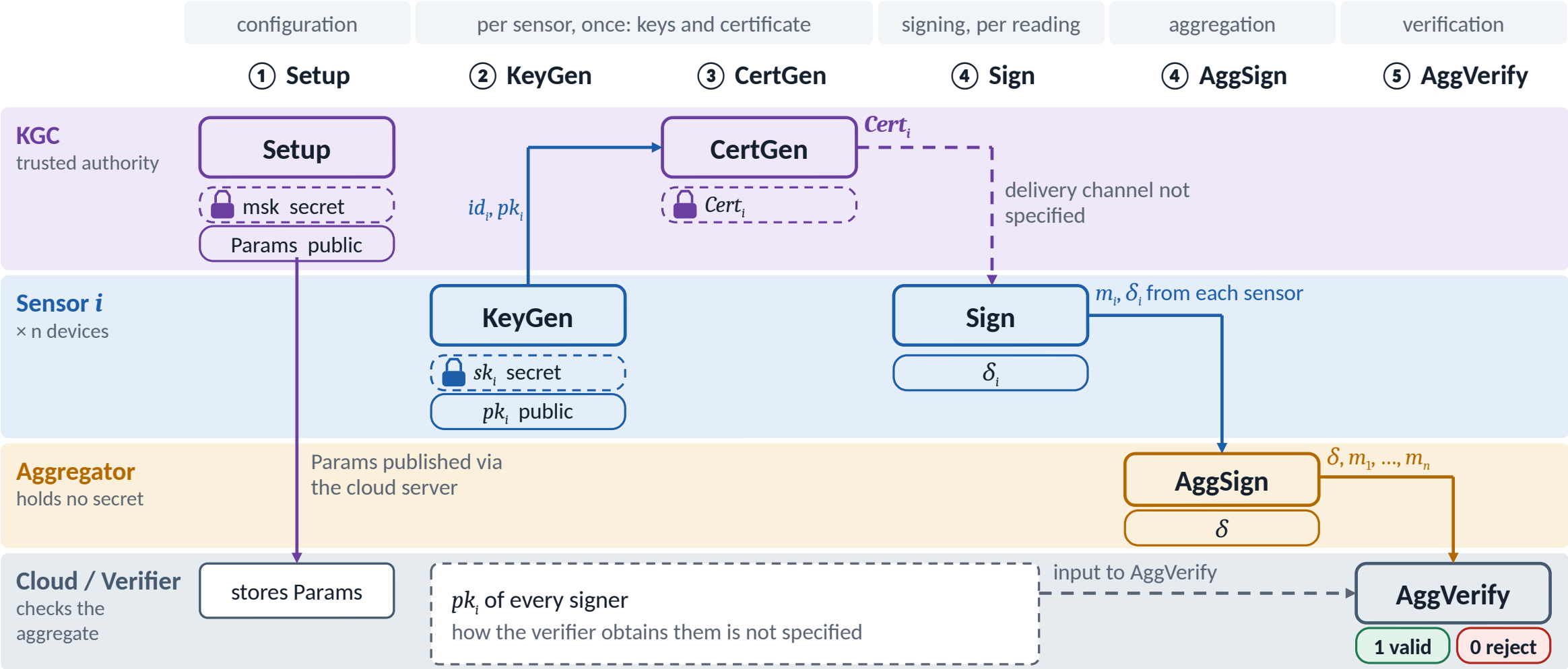
EXPERIMENT

we measured it

OUR AUDIT

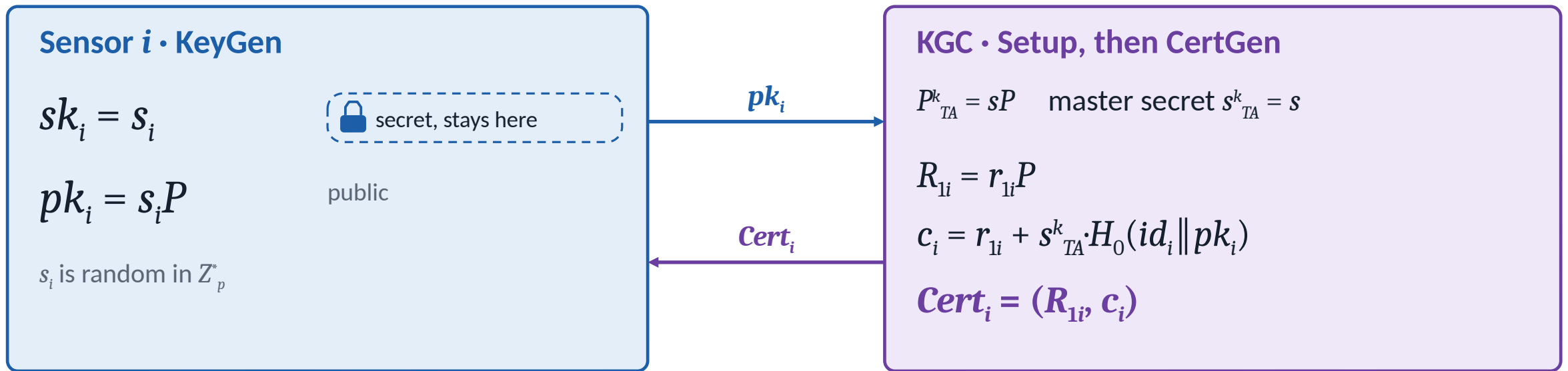
our own finding

Six algorithms, five phases, four parties



Verma et al.: every sensor gets a certificate from the KGC

P generates a prime-order group G · aP is P added to itself a times · H_0 hashes into Z_p^* · id_i is the sensor's identity



Anyone can check a certificate against public values: $c_i P = R_{1i} + H_0(id_i \| pk_i) \cdot P^k_{TA}$

The KGC knows $Cert_i$ because it made it. Only the sensor knows sk_i .

Signing mixes a fresh nonce, the certificate and the secret key

$$R_i = R_{1i} + r_{2i}P$$

$$v_i = H_1(m_i \| pk_i \| id_i \| \Delta)$$

$$z_i = r_{2i} + c_i + sk_i v_i$$

$$\delta_i = (R_i, z_i)$$

WHO KNOWS EACH TERM OF z_i

r_{2i} fresh nonce
sensor only

c_i certificate
the KGC knows it too

$sk_i v_i$ secret key $\times$ hash
 sk_i sensor only; v_i anyone

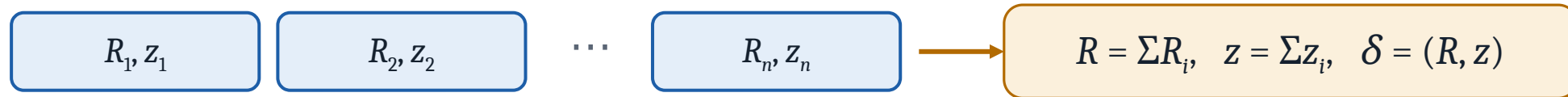
WHAT GOES INTO THE HASH v_i



v_i never sees R_i or r_{2i} .

Verification, and the appeal of a constant-size aggregate

AGGREGATION



one point and one scalar, for any number of signers (paper Table VI)

AGGVERIFY (CALLED “VERY” IN THE PAPER'S SECTIONS IV-V)

accept iff $zP = R + (\sum h_0^i) \cdot P_{TA}^k + \sum v_i \cdot pk_i$

with $h_0^i = H_0(id_i \| pk_i)$ and $v_i = H_1(m_i \| pk_i \| id_i \| \Delta)$ recomputed for every signer

“... to obtain an aggregate signature with the fixed length, Verma et al. remove the random value selected by the signature algorithm from the calculation of hash function. However, due to this reason, the above construction is insecure.” — Qiao et al., §V-D

The compact aggregate comes from keeping R_i out of the hash.

The attacker: a KGC that has gone bad

Legitimate sensor

id_i

signs a reading m_i

valid signature
 $\delta_i = (R_i, z_i)$

The KGC sees this one signature.

THE KGC KNOWS

- ✓ master secret $s_{TA}^k = s$
- ✓ the certificate it issued: R_{ii}, c_i
- ✓ public: $id_i, pk_i, \Delta, P_{TA}^k$
- ✓ one valid pair: m_i and (R_i, z_i)

PRESENTER CUE

KGC does NOT know sk_i or r_{2i} .

THE KGC DOES NOT KNOW



sk_i

the sensor's secret key



r_{2i}

this signature's nonce

Goal: a valid δ'_i on a new message m'_i , for the same identity id_i .

The KGC holds everything except the two secrets inside z_i .

The puzzle

PRESENTER CUE

Pause. Answer: rescale, don't open.

How can the KGC forge a new signature if it does NOT know r_{2i} or sk_i ?

WHAT THE KGC CAN COMPUTE

$$r_{2i}P = R_i - R_{1i} \quad \text{a point}$$

$$Z_i - C_i = \text{🔒} \quad r_{2i} + sk_i v_i$$

WHAT A FORGERY ON m'_i WOULD NEED

a matching commitment and response
for a new hash value v'_i
with sk_i and r_{2i} still unknown

Hint: it does not have to open the sealed number. It only has to rescale it.

Step 1: strip off the certificate

PRESENTER CUE

Step 1: subtract the certificate.

THE ATTACK, IN FIVE STEPS

① strip the certificate

② normalise by v_i

③ choose m'_i , get v'_i

④ build R'_i

⑤ build z'_i

The KGC issued R_{1i} and c_i , so it can subtract them:

$$r_{2i}P = R_i - R_{1i}$$

a group element: the KGC can compute it, but cannot take its discrete logarithm

$$r_{2i} + sk_i v_i = z_i - c_i$$

two unknowns in one number: it stays sealed

where $v_i = H_1(m_i \| pk_i \| id_i \| \Delta)$ can be computed by anyone

Known: the point $r_{2i}P$. **Sealed:** the number $r_{2i} + sk_i v_i$.

Step 2: normalise by v_i

PRESENTER CUE

Step 2: normalise by v_i .

THE ATTACK, IN FIVE STEPS

① strip the certificate

② **normalise by v_i** ③ choose m'_i , get v'_i ④ build R'_i ⑤ build z'_i

Divide by v_i : it is public, and non-zero, so it can be inverted in a prime-order group.

$$\alpha = (z_i - c_i)/v_i = \overbrace{r_{2i}/v_i + sk_i}^{\text{locked}}$$

α is a number the KGC holds; what it is made of stays sealed

$$A = (R_i - R_{1i})/v_i = (r_{2i}/v_i)P$$

A is a group element the KGC holds

Both are scaled by $1/v_i$. Neither reveals r_{2i} or sk_i on its own.

Step 3: rescale to any message

THE ATTACK, IN FIVE STEPS

① strip the certificate



② normalise by v_i



③ choose m'_i , get v'_i



④ build R'_i



⑤ build z'_i

$$v'_i = H_1(m'_i \| pk_i \| id_i \| \Delta)$$

$$\begin{aligned} R'_i &= R_{1i} + v'_i \cdot A \\ &= R_{1i} + (v'_i r_{2i} / v_i) P \end{aligned}$$

$$\begin{aligned} z'_i &= v'_i \cdot \alpha + c_i \\ &= v'_i r_{2i} / v_i + c_i + sk_i v'_i \end{aligned}$$

$$\delta'_i = (R'_i, z'_i)$$

PRESENTER CUE

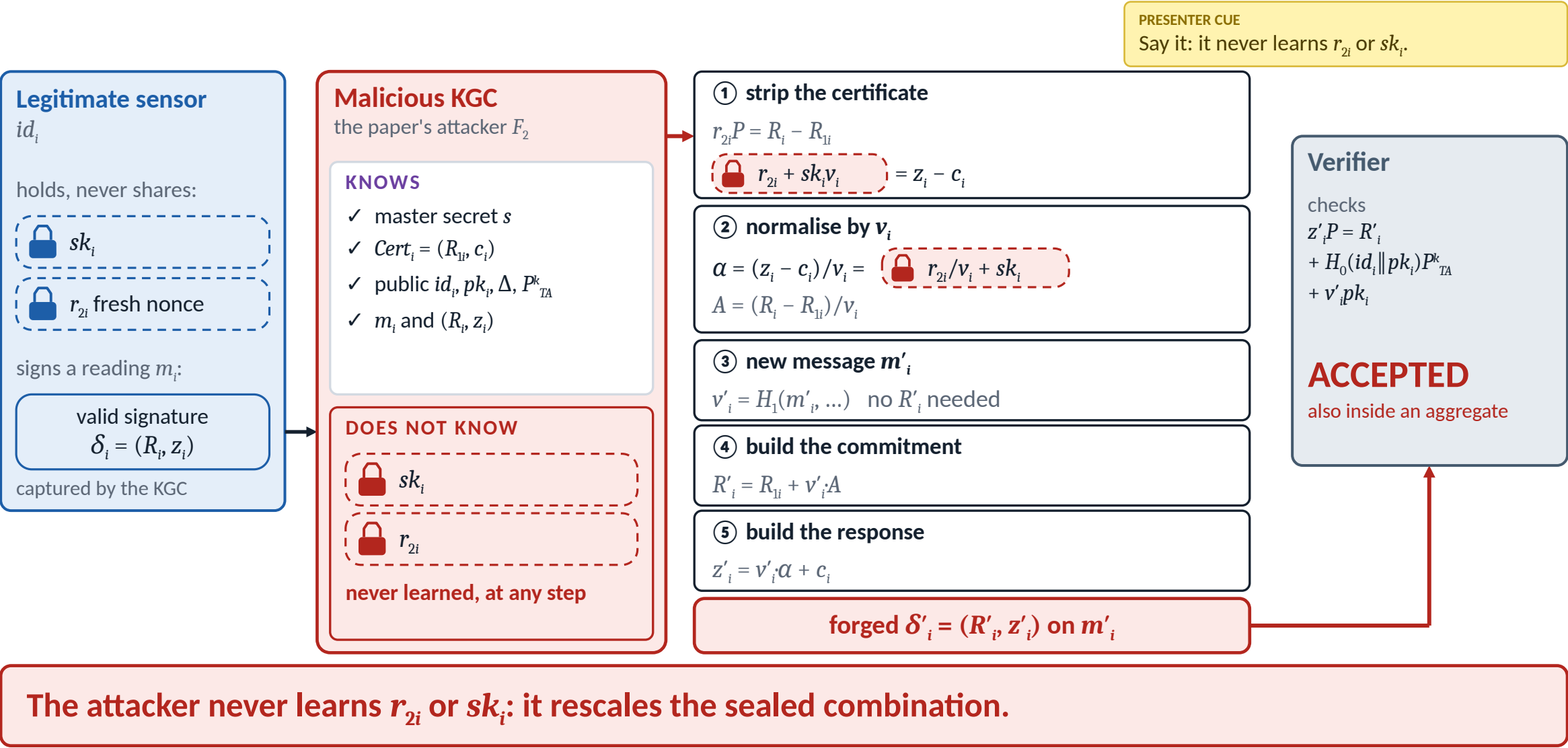
New $v'_i \rightarrow R'_i \rightarrow z'_i \rightarrow \text{VALID}$

computable now:
 R'_i is not an input

looks honest: fresh nonce $v'_i r_{2i} / v_i$

The result has exactly the form of an honest signature.

The forgery verifies, and the sealed values were never opened



Demo 1: we run the attack on Verma et al.

PRESENTER CUE

Terminal fails? → Backup B11

```
$ make attack
```

WHAT TO POINT AT

- 1 The honest signature verifies.
- 2 The attacker is the KGC; it does not know sk_i .
- 3 The forged signature verifies, even inside an aggregate.

```
$ make attack
```

```
TARGET 1 Verma et al., CB-CAS (IEEE IoT-J 2020)
```

```
honest message      : temp=21.4C;line=A;seq=17
```

```
honest sig verifies : True
```

```
attacker is the KGC: holds the master key and  
the certificate, has seen ONE signature.
```

```
It does NOT know sk_i.
```

```
target message      : temp=-40.0C;line=A;seq=17;SHUTDOWN
```

```
forged sig verifies : True    <-- FORGERY SUCCEEDED
```

```
forgery survives aggregation : True
```

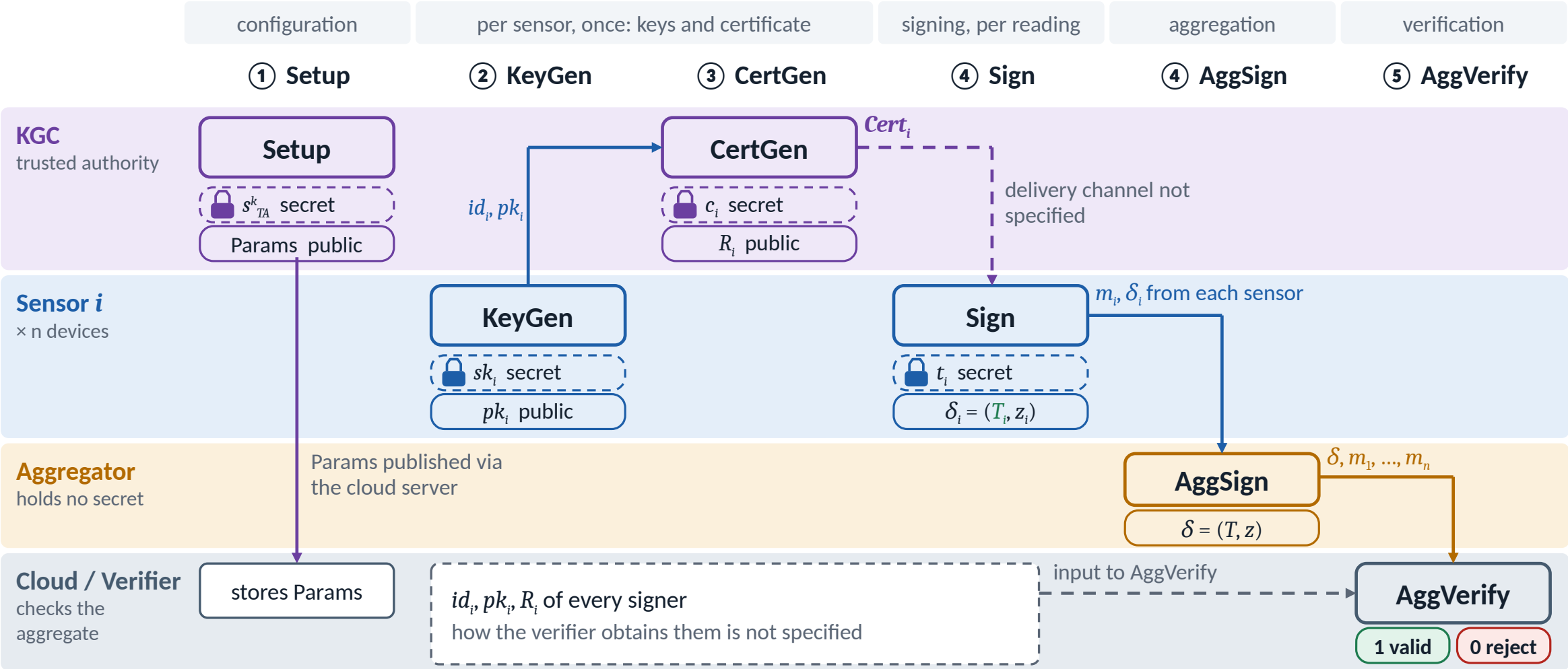
8 / 8 forgeries verify: 4 messages × 2 backends

8 / 8 also pass AggVerify with an honest signature

the forging function takes no secret-key argument

Reproduced: one observed signature plus the issued certificate forges messages of the attacker's choice.

Qiao et al.: the same six algorithms, with new objects



What Qiao et al. changed

	VERMA ET AL.	QIAO ET AL.
Certificate	$c_i = r_{1i} + s_{TA}^k H_0(id_i pk_i)$	$c_i = r_i + s_{TA}^k H_0(id_i pk_i R_i), \text{ } R_i \text{ public}$
Commitment	$R_i = R_{1i} + r_{2i}P$	$T_i = t_iP$
Hash inputs	$v_i = H_1(m_i pk_i id_i \Delta)$	$v_i = H_1(m_i pk_i R_i id_i T_i \Delta)$ $u_i = H_2 \text{ of the same input}$
Response	$z_i = r_{2i} + c_i + sk_i v_i$	$z_i = t_i + c_i u_i + sk_i v_i$
Aggregate	$(R, z) = (\sum R_i, \sum z_i) \text{ one point + one scalar}$	$(T, z), \text{ } T = (T_1, ..., T_n), \text{ } z = \sum z_i$
Verification	$zP = R + (\sum h_0^i)P_{TA}^k + \sum v_i pk_i$	$zP = \sum T_i + \sum u_i R_i + (\sum u_i h_0^i)P_{TA}^k + \sum v_i pk_i$

Typos in §V-A, corrected in our code: KeyGen prints $pk_i = s_i$ (we use s_iP); the certificate check prints R_{1i} (we use R_i).

Several things change. For this attack, what matters is that T_i now enters the hashes.

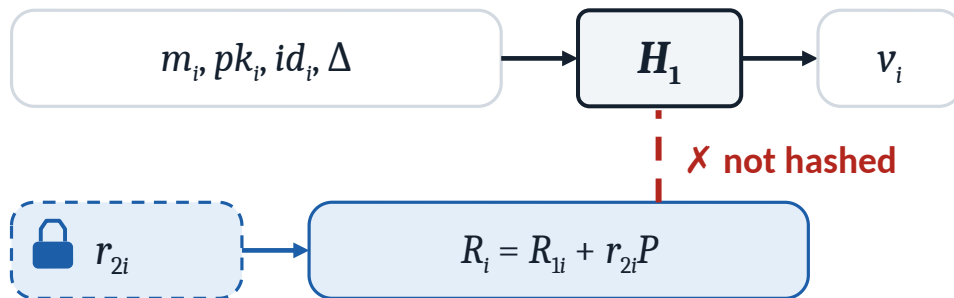
Old vs new: is the commitment coupled to the hash?

PRESENTER CUE

Key idea: T_i enters the hash.

OLD • Verma et al.

message and commitment are not coupled through the hash



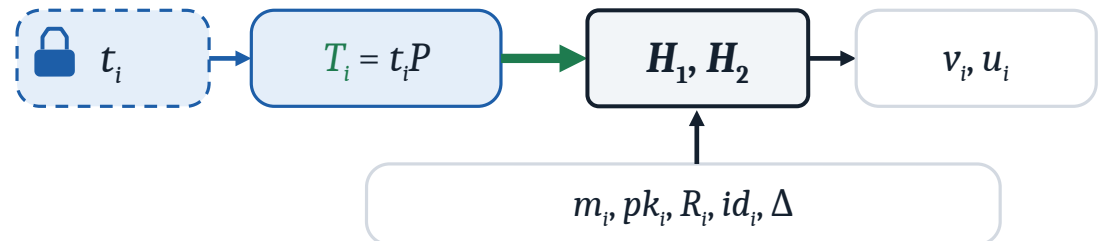
THE ATTACKER'S ORDER

- ① pick any m'_i
- ② compute v'_i , which needs no R'_i
- ③ build R'_i and z'_i

forgery verifies

NEW • Qiao et al.

message and commitment are both inputs to the hash



THE RESCALING ATTACK NOW NEEDS BOTH AT ONCE



circular

 T'_i affects v'_i , and v'_i affects T'_i

the old direct rescaling route is broken

This explains why the reproduced rescaling route no longer applies. Formal security is the paper's EUF-CMA proof.

Demo 2: the same strategy against the repaired scheme

PRESENTER CUE

Terminal fails? → Backup B11

\$ make attack

WHAT TO POINT AT

- 1 The honest signature still verifies.
- 2 Same strategy, adapted: strip $c_i u_i$, not c_i .
- 3 16 candidates T'_i , all different, none verifies.

(scroll to Target 2)

```
TARGET 2 Qiao et al., repaired CBAS (IEEE Syst. J. 2023)
honest sig verifies : True
forgery succeeded   : False
iterations tried    : 16
candidate T' per iteration (first 8 bytes):
  1. 7a30fa1d6d0e2eb4... (values differ per run)
  2. 26a9b48630a0b741...
  ... 10 more, all distinct
```

PAPER

proves EUF-CMA security

under the discrete logarithm assumption, in the random-oracle model (Theorem 1)

EXPERIMENT

this implementation

does not admit this same algebraic forgery route:
0 / 8 attempts succeed

OUR AUDIT

not shown here

a proof of security: one failed attack is evidence
about one attack

The repaired implementation does not admit this forgery route. That is evidence, not a proof of security.

A controlled experiment: remove only the T_i binding

SCHEME	WHAT v_i IS HASHED FROM	COMMITMENT IN THE HASH?
A • Verma et al.	m_i, pk_i, id_i, Δ	✗ no: R_i is not hashed
B • Qiao et al.	$m_i, pk_i, R_i, id_i, T_i, \Delta$	✓ yes: T_i is hashed
C • Qiao, T_i removed	$m_i, pk_i, R_i, id_i, \Delta$	✗ no: T_i removed

EVERYTHING ELSE IN C IS THE REPAIRED CODE

- same setup, keys, certificate, aggregation, verification
- R_i still hashed; u_i and v_i still separate

CONTROLS: C IS A WORKING SCHEME

- honest signatures verify, in 8 / 8 trials
- a different message is rejected, in 8 / 8 trials

Between B and C, only the T_i binding differs.

Remove the T_i binding, and the attack returns

PRESENTER CUE

Only the T_i binding is removed.

A Original Verma construction

$$v_i = H_1(m_i \| pk_i \| id_i \| \Delta)$$

ATTACK SUCCEEDS

8 / 8 forgeries verify

B Qiao repaired construction

$$v_i = H_1(\dots \| R_i \| id_i \| T_i \| \Delta)$$

ATTACK FAILS

0 / 8 verify; 16 distinct candidates each

C Qiao with the T_i binding removed

$$v_i = H_1(\dots \| R_i \| id_i \| \Delta)$$

ATTACK SUCCEEDS

8 / 8 forgeries verify

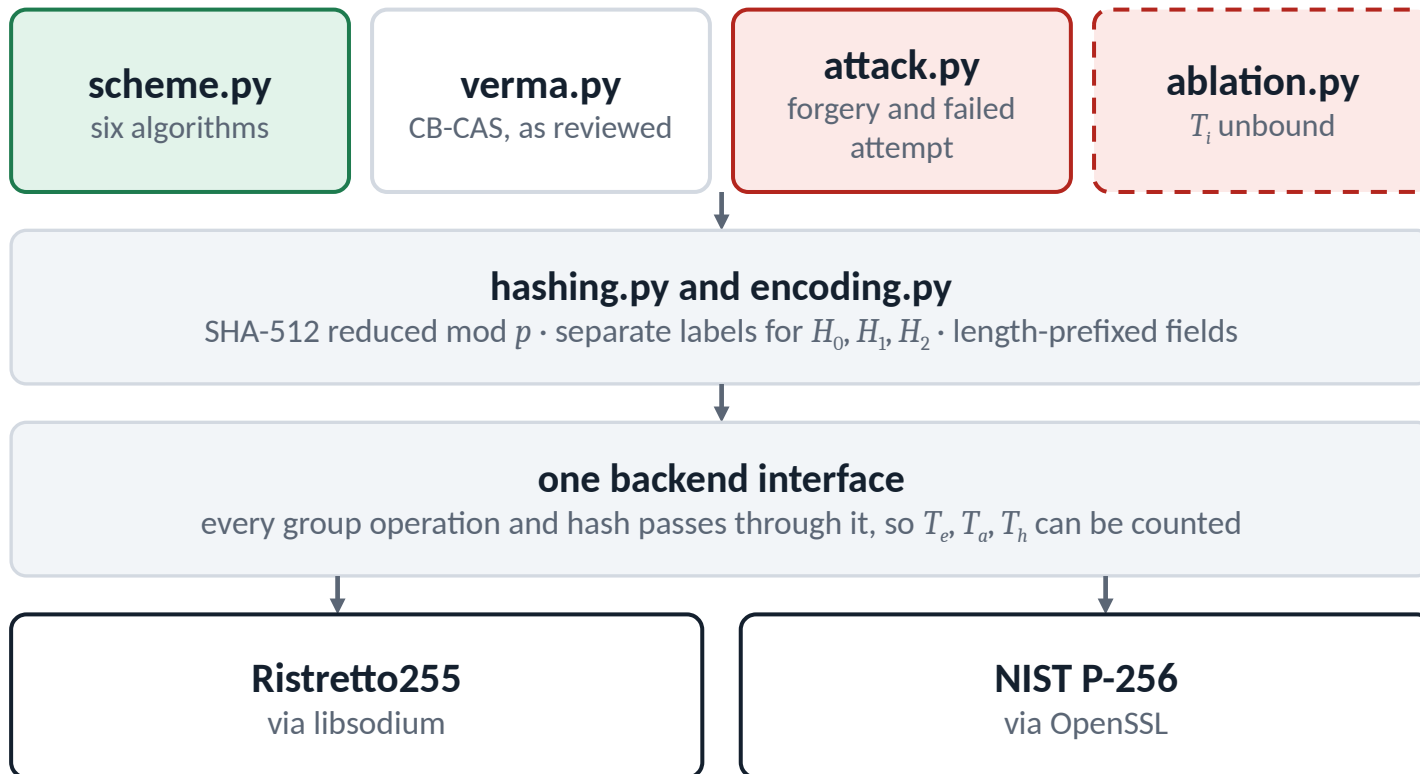
```
$ .venv/bin/python -m pytest tests/test_forgery.py -k ablat -vv 10 passed fallback: B12
```

Shown: the T_i binding is necessary against this attack. Not shown: that the other changes are unnecessary (R_i in H_0 targets a different attack), or that the full scheme is secure.

The ablation isolates the T_i binding as the critical difference for blocking the reproduced rescaling attack.

How we built it, and how we checked it

THE CODE, LAYER BY LAYER

**62**

self-test checks, 31 per backend

221tests: 101×2 backends + 19**=**

identical operation counts and accept/reject outcomes on both backends

Every result in this talk comes from code that passes the same checks on two independent group implementations.

The paper's claim: security at no extra cost

SCHEME	SIGN • TABLE II	VERIFY • TABLE II	AGGSIGN • TABLE III	AGGVERIFY • TABLE III	
Verma et al. [11]	$1T_e + 1T_h$	$3T_e + 2T_a + 2T_h$	$(n-1)T_a$	$(n+2)T_e + (n+1)T_a + 2nT_h$	identical
Qiao et al. “Our CBS / CBAS scheme”	$1T_e + 1T_h$	$3T_e + 2T_a + 2T_h$	$(n-1)T_a$	$(n+2)T_e + (n+1)T_a + 2nT_h$	

PAPER • SECTION I-D

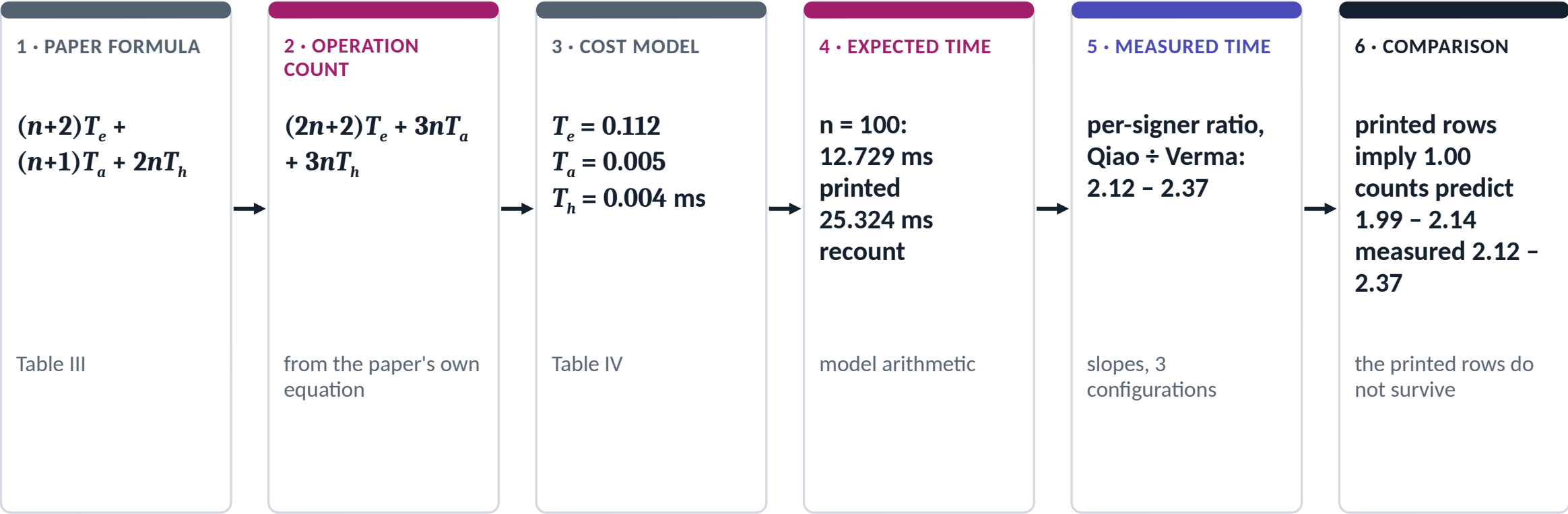
“... our proposal has the same high computational efficiency as Verma et al.'s CBAS scheme, but our scheme is more secure.”

T_e : scalar multiplication in G · T_a : point addition in G · T_h : one hash · paper's Table IV: 0.112, 0.005, 0.004 ms

The repaired Sign computes two hashes, and verification gains a term per signer. Can the cost still be identical?

From the paper's formula to a measurement

PRESENTER CUE
Equation → count → cost → measure.



We do not jump from “12.729 ms” to “the paper is wrong”. Every step can be checked on its own.

Counting the operations in the paper's own equation

TERM OF THE EQUATION	T_e	T_a
zP	1	-
ΣT_i	0	$n-1$
$\Sigma u_i R_i$	n	$n-1$
$(\Sigma u_i h_0^i) P_{TA}^k$	1	-
$\Sigma v_i p k_i$	n	$n-1$
combine the four terms	-	3
total	$2n+2$	$3n$

hashes per signer: $h_0^i, v_i, u_i \rightarrow 3nT_h$

	PAPER	RECOUNT
Sign	$1T_e + 1T_h$	$1T_e + 2T_h$
Verify	$3T_e + 2T_a + 2T_h$	$4T_e + 3T_a + 3T_h$
AggSign	$(n-1)T_a$	no group operation
AggVerify	$(n+2)T_e + (n+1)T_a + 2nT_h$	$(2n+2)T_e + 3nT_a + 3nT_h$

Calibration. The same counter reproduces Verma's printed AggVerify, AggSign and Verify rows exactly: the discrepancy is in the printed row, not in the method.

The paper's own equation needs two scalar multiplications per signer, $u_i R_i$ and $v_i p k_i$, not one.

We tested the obvious objection: caching

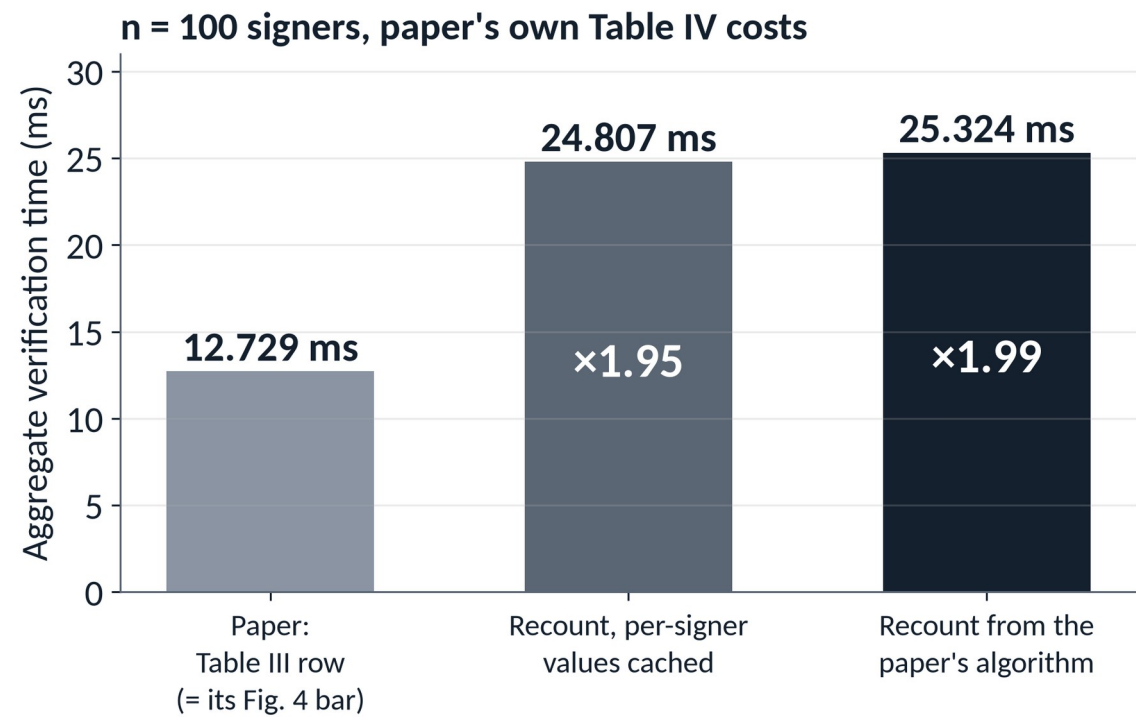
CONVENTION	QIAO • ONE VERIFY	VERMA • ONE VERIFY	QIAO • AGGVERIFY T_e
No caching: the convention behind Verma's printed row	$4T_e + 3T_a + 3T_h$	$3T_e + 2T_a + 2T_h$ = printed row	$2n + 2$
Cache every per-signer constant: h_0^i and $C_i = R_i + h_0^i P_{TA}^k$ (Verma: $h_0^i P_{TA}^k$)	$3T_e + 2T_a + 2T_h$ = printed Table II row	$2T_e + 2T_a + 1T_h$	$2n + 1$

OUR AUDIT

Aggregate, n = 100, with caching
 $201T_e + 299T_a + 200T_h$ against the printed $102T_e + 101T_a + 200T_h$: T_h matches, T_e does not.
Folding $\sum u_i R_i$ needs the secret c_i ; multi-scalar multiplication is a different cost model (backup B8).

Caching reproduces the single-signature row, but aggregate verification still needs at least $2n+1$ scalar multiplications, against the printed $n+2$.

At n = 100 the paper's own costs give about twice the printed time



Source: paper Tables III–IV; recount from the implementation (audit/verify_claims.py). Model values, not measurements.

PAPER

12.729 ms is the bar in the paper's own Fig. 4
So Fig. 4 was computed from Table III, and carries its count.

OUR AUDIT

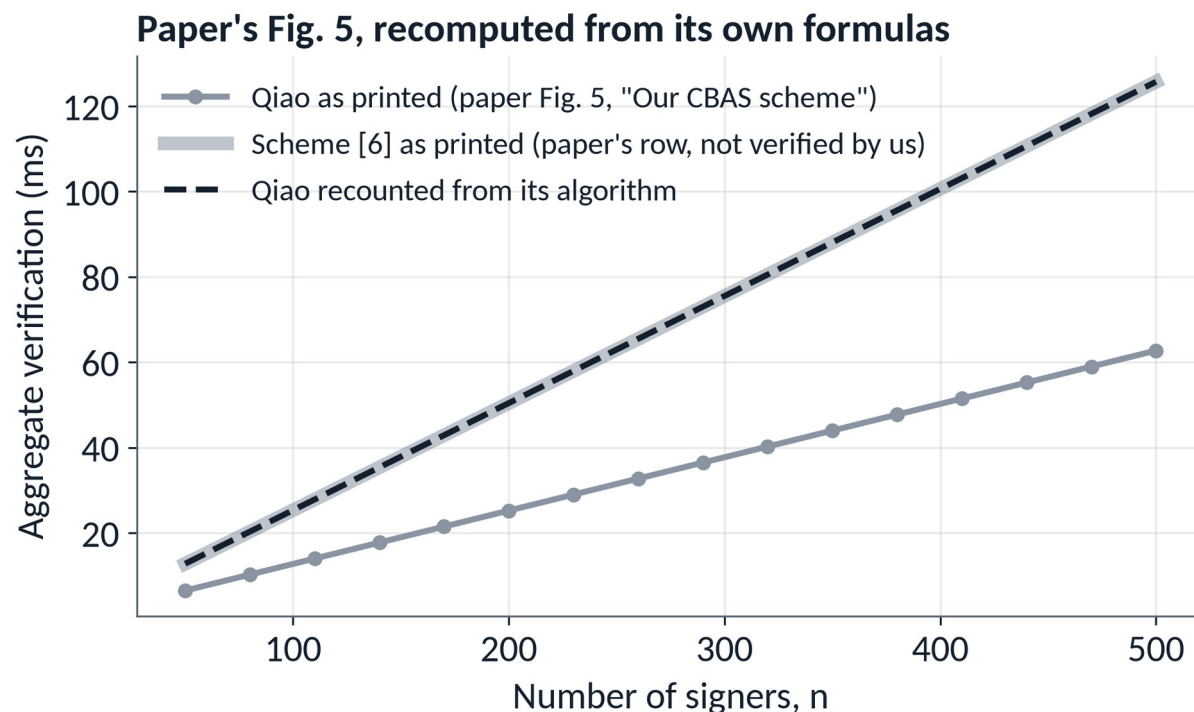
×1.99 from the paper's algorithm; ×1.95 even with caching
Per signer under Table IV: 125 → 251 μs (×2.01).

OUR AUDIT

Not a one-off typo
A printed row that reproduces exactly in a figure, yet does not follow from the printed algorithm.

The printed row agrees with Fig. 4 and disagrees with the printed verification equation.

The paper's figures and its comparison with [6] inherit the count



Source: paper Tables III–IV formulas, $n = 50 \dots 500$ (step 30); recount from the implementation. Model values.

OUR AUDIT

Figs. 3–5 are table formulas × Table IV

Figs. 3 and 4: every label reproduces exactly. Fig. 5: line ends match, read from the plot.

PAPER

Using the paper's own row for [6]

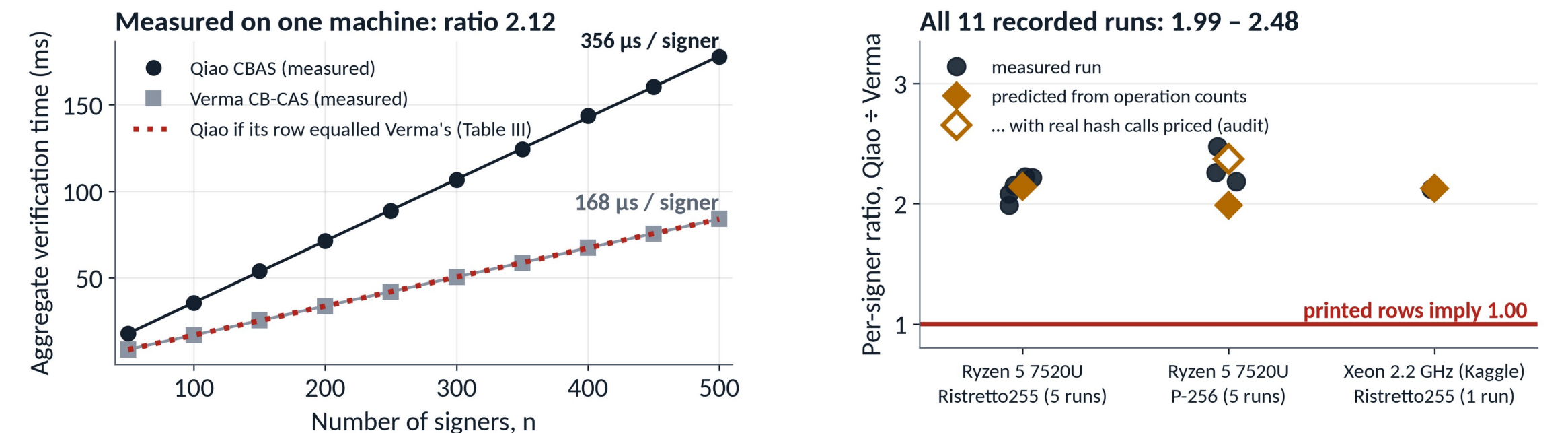
25.324 ms (Qiao recount) vs 25.319 ms ([6]) at $n = 100$. The $\approx 49\%$ advantage in Fig. 4 (13.224 vs 25.814 ms) is not supported.

We have not verified scheme [6]. No claim about its real speed.

Unchanged: bandwidth, $n|G|$ vs $2n|G|$ (paper Table VI).

Only from the paper's own numbers: its computational advantage over [6] disappears; its bandwidth advantage does not.

Independent measurements are consistent with the higher cost



Source: bench/results/sweep_kaggle-xeon.json — Xeon 2.2 GHz, Ristretto255/libsodium, n = 50 ... 500 (10 points), median of 7 interleaved rounds.

Source: bench/results/sweep_ryzen5-7520u.json (10 runs), sweep_kaggle-xeon.json (1 run); real-hash prediction: audit/dagger_checks.py.

P-256: pricing the real hash calls (point serialisation) lifts the prediction from 1.99 to ≈ 2.37 (our audit). Ratios only: absolute times are not comparable with the paper's.

Counting and timing both point the same way: per-signer cost is about twice Verma's, not equal.

What we established, and what we did not

PRESENTER CUE

3 things to remember.

Reproduced from the paper

- A malicious KGC forges Verma signatures from one observed signature (§IV-B): 8 / 8, two backends
- The same strategy fails against Qiao's repaired scheme: 0 / 8

① how the KGC forges without r_{2i} or sk_i

Our original audit

- Removing only the T_i binding brings the attack back: 8 / 8
- Table III's AggVerify row does not follow from the algorithm: about $\times 2$ at $n = 100$; Fig. 4 repeats it
- Timing: per-signer ratio 2.12 – 2.37, where the printed rows imply 1.00

② why binding T_i changes the algebra

Not established

- Formal security of the repaired scheme: the paper's proof, not audited
- Fidelity to Verma's original paper: we implemented Qiao's description
- Anything about scheme [6]'s real cost

③ why the cost rows do not follow from the algorithm

Next, as questions only: does the repaired scheme have weaknesses of its own that its proof does not rule out in practice?

Thank you

Questions?

BACKUP • ONLY IF ASKED

Appendix

B1 The complete Verma scheme

B2 The complete Qiao scheme

B3 The full malicious-KGC derivation

B4 Correctness of aggregate verification

B5 Security models and the paper's proof

B6 Why T_i is in both hashes

B7 The complete operation-count table

B8 Caching and precomputation

B9 Benchmark methodology

B10 Testing and reproducibility

B11 Demo fallback: Demos 1 and 2

B12 Demo fallback: Demo 3

B1 • The complete Verma scheme

ALGORITHM	DEFINITION
Setup	$s \leftarrow_R Z_p^*, s_{TA}^k = s, P_{TA}^k = sP$ $Params = (G, p, P, H_0, H_1, P_{TA}^k, \Delta)$
KeyGen	$s_i \leftarrow_R Z_p^*, sk_i = s_i, pk_i = s_iP$
CertGen	$r_{1i} \leftarrow_R Z_p^*, R_{1i} = r_{1i}P, c_i = r_{1i} + s_{TA}^k H_0(id_i pk_i)$ $Cert_i = (R_{1i}, c_i), \text{ checked by } c_iP = R_{1i} + H_0(id_i pk_i)P_{TA}^k$
Sign	$r_{2i} \leftarrow_R Z_p^*, R_i = R_{1i} + r_{2i}P, v_i = H_1(m_i pk_i id_i \Delta)$ $z_i = r_{2i} + c_i + sk_i v_i, \delta_i = (R_i, z_i)$
AggSign	$R = \Sigma R_i, z = \Sigma z_i, \delta = (R, z)$
AggVerify	$h_0^i = H_0(id_i pk_i), v_i = H_1(m_i pk_i id_i \Delta)$ accept iff $zP = R + (\Sigma h_0^i)P_{TA}^k + \Sigma v_i pk_i$

Source: Qiao et al., §IV-A. Code: verma.py. “Very” is the paper's name for AggVerify.

B2 • The complete Qiao scheme

ALGORITHM	DEFINITION
Setup	$s \leftarrow_R Z_p^*, s_{TA}^k = s, P_{TA}^k = sP$ $Params = (G, p, P, H_0, H_1, H_2, P_{TA}^k, \Delta)$
KeyGen	$s_i \leftarrow_R Z_p^*, sk_i = s_i, pk_i = s_iP$
CertGen	$r_i \leftarrow_R Z_p^*, R_i = r_iP, c_i = r_i + s_{TA}^k H_0(id_i pk_i R_i)$ $Cert_i = (c_i, R_i), R_i \text{ public; check } c_iP = R_i + H_0(id_i pk_i R_i)P_{TA}^k$
Sign	$t_i \leftarrow_R Z_p^*, T_i = t_iP, v_i = H_1(m_i pk_i R_i id_i T_i \Delta)$ $u_i = H_2(m_i pk_i R_i id_i T_i \Delta), z_i = t_i + c_iu_i + sk_iv_i, \delta_i = (T_i, z_i)$
AggSign	$z = \Sigma z_i, \delta = (T, z) \text{ with } T = (T_1, ..., T_n)$
AggVerify	$h_0^i = H_0(id_i pk_i R_i), v_i, u_i \text{ recomputed as in Sign}$ accept iff $zP = \Sigma T_i + \Sigma u_iR_i + (\Sigma u_ih_0^i)P_{TA}^k + \Sigma v_i pk_i$

Printed typos corrected in our code: KeyGen prints $pk_i = s_i$ (we use s_iP); the certificate check prints R_{ii} (we use R_i). Notation $\rightarrow$ code: $s_{TA}^k \rightarrow s \cdot P_{TA}^k \rightarrow pk_ta \cdot h_0^i \rightarrow h0 \cdot \Delta \rightarrow delta \cdot$
“Very” $\rightarrow$ agg_verify.

B3 • The full malicious-KGC derivation

STEPS (2a)–(2d), PAPER §IV-B

$$(a) \ r_{2i}P = R_i - R_{1i}, \quad r_{2i} + sk_i v_i = z_i - c_i$$

$$(b) \ \alpha = (1/v_i)(z_i - c_i) = r_{2i}/v_i + sk_i$$

$$A = (1/v_i)(R_i - R_{1i}) = (r_{2i}/v_i)P$$

$$(c) \ v'_i = H_1(m'_i \| pk_i \| id_i \| \Delta)$$

$$R'_i = R_{1i} + v'_i A = R_{1i} + (v'_i r_{2i}/v_i)P$$

$$z'_i = v'_i \alpha + c_i = v'_i r_{2i}/v_i + c_i + sk_i v'_i$$

$$(d) \ \text{output } \delta'_i = (R'_i, z'_i)$$

PAPER STEP	attack.py • forge_verma
(a) v_i and $1/v_i$	lines 73–74
(b) α, A	lines 77–78
(c) v'_i, R'_i, z'_i	lines 81–83

(3) IT VERIFIES

$$\begin{aligned} z'_i P &= (v'_i r_{2i}/v_i)P + c_i P + sk_i v'_i P \\ &= (v'_i r_{2i}/v_i)P + R_{1i} + H_0(id_i \| pk_i)P_{TA}^k + v'_i pk_i \\ &= R'_i + H_0(id_i \| pk_i)P_{TA}^k + v'_i pk_i \end{aligned}$$

The same attack in one line, $\lambda = v'_i/v_i$ (our rewriting)

$$R'_i = R_{1i} + \lambda(R_i - R_{1i})$$

$$z'_i = c_i + \lambda(z_i - c_i)$$

The function takes no secret key; a test checks its signature.

B4 • Why aggregate verification accepts honest signatures

QIAO ET AL.

$$\begin{aligned}
 zP &= \Sigma(t_i + c_i u_i + sk_i v_i)P \\
 &= \Sigma t_i P + \Sigma u_i (r_i + s_{TA}^k h_0^i)P + \Sigma sk_i v_i P \\
 &= \Sigma T_i + \Sigma u_i R_i + (\Sigma u_i h_0^i)P_{TA}^k + \Sigma v_i pk_i
 \end{aligned}$$

VERMA ET AL.

$$\begin{aligned}
 zP &= \Sigma(r_{2i}P + c_i P + v_i pk_i), \quad \text{with } c_i P = R_{1i} + h_0^i P_{TA}^k \text{ and } R_i = R_{1i} + r_{2i}P \\
 &= \Sigma(R_i + h_0^i P_{TA}^k + v_i pk_i) = R + (\Sigma h_0^i)P_{TA}^k + \Sigma v_i pk_i
 \end{aligned}$$

B5 • The two adversaries and the paper's proof

ADVERSARY	CAN	CANNOT
F_1 malicious user	replace the public key of any user; ask for keys, certificates and signatures	know the master key; replace pk^* of the challenge identity; ask for $Cert^*$
F_2 malicious KGC	know the master key, so every certificate; ask for keys and signatures	replace public keys; ask for the secret key sk^* of the challenge identity

PAPER

Theorem 1: if the discrete logarithm problem is hard, the CBAS construction is EUF-CMA secure (random-oracle model).

Lemma 2 (F_1): fork on H_0 . Advantage $\geq (1 - 1/e) \cdot \epsilon_1 / (e \cdot q_0(x_1 + x_2 + 1))$

Lemma 3 (F_2): fork on H_1 . Advantage $\geq (1 - 1/e) \cdot \epsilon_2 / (e \cdot q_h(x_1 + x_3 + 1))$

Aggregate security follows because AggSign uses no secret material.

We did not audit these proofs, and noticed inconsistencies in the lemma texts (for example, Lemma 3's H_1 table omits T_i).

B6 · Why T_i appears in both hashes

WHERE T_i IS HASHED	KGC RESCALING ATTACK	MIRROR ATTACK BY A MALICIOUS USER
H_1 only	blocked	forges
H_2 only	forges	blocked
both: Qiao et al.	blocked	blocked
neither: the ablation	forges	forges

- The KGC attack scales the term $sk_i v_i$, so it needs v'_i before T'_i : it works unless T_i is in H_1 .
- The mirror attack (a user who knows sk_i but not c_i) scales $c_i u_i$: it works unless T_i is in H_2 .
- A variant with one hash for both coefficients, T_i bound, still blocked the KGC attack, so we do not claim the split is needed against it.
- With T_i in H_1 , rescaling $T'_i = \lambda T_i$ needs $H_1(m'_i \| pk_i \| R_i \| id_i \| \lambda T_i \| \Delta) = \lambda v_i$: a search over hash outputs.

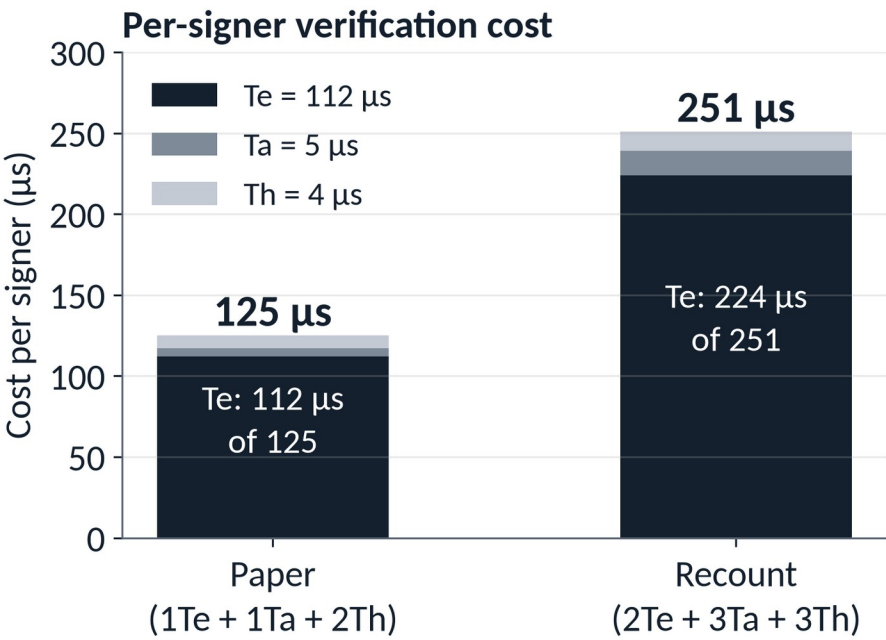
Source: presentation/audit/dagger_checks.py, both backends; not in the repository's test-suite.

B7 • The complete operation-count table

QIAO ET AL.	PAPER	COUNTED
Sign	$1T_e + 1T_h$	$1T_e + 2T_h$
Verify	$3T_e + 2T_a + 2T_h$	$4T_e + 3T_a + 3T_h$
AggSign	$(n-1)T_a$	0 group ops
AggVerify	$(n+2)T_e + (n+1)T_a + 2nT_h$	$(2n+2)T_e + 3nT_a + 3nT_h$

VERMA ET AL.	PAPER	COUNTED
Sign	$1T_e + 1T_h$	$1T_e + 1T_a + 1T_h$
Verify	$3T_e + 2T_a + 2T_h$	same ✓
AggSign	$(n-1)T_a$	same ✓
AggVerify	$(n+2)T_e + (n+1)T_a + 2nT_h$	same ✓

Counted on both backends (identical); Verma's printed Sign omits one T_a (negligible).



Source: paper Table IV unit costs × operation counts per signer (ratio 2.008). Model values.

B8 • Caching and precomputation, in full

CONVENTION	QIAO • ONE VERIFY	VERMA • ONE VERIFY	QIAO • AGGVERIFY, n = 100
none	$4T_e + 3T_a + 3T_h$	$3T_e + 2T_a + 2T_h$	$202T_e + 300T_a + 300T_h$
cache h^i_0 only (derived)	$4T_e + 3T_a + 2T_h$	$3T_e + 2T_a + 1T_h$	$202T_e + 300T_a + 200T_h$
cache C_i (Verma: $h^i_0 P^k_{TA}$)	$3T_e + 2T_a + 2T_h$	$2T_e + 2T_a + 1T_h$	$201T_e + 299T_a + 200T_h$
printed (Tables II and III)	$3T_e + 2T_a + 2T_h$	$3T_e + 2T_a + 2T_h$	$102T_e + 101T_a + 200T_h$

- $C_i = R_i + h^i_0 P^k_{TA}$ equals $c_i P$: static per signer, so a verifier serving a fixed fleet could store it.
- Folding: $\sum u_i R_i + (\sum u_i h^i_0) P^k_{TA} = (\sum u_i c_i) P$ is one multiplication, but needs the secret c_i .
- Multi-scalar multiplication: a different cost model; it applies to both schemes.

B9 • Benchmark methodology and the P-256 gap

- Warm the CPU to a steady clock first: it ramps $\approx 2\times$ under load, which once made $n = 100$ look cheaper than $n = 50$
- Time every workload (both schemes, every n) once per round, interleaved; median of 7 rounds
- Calibrate repetitions so each sample lasts ≥ 20 ms; report medians
- Fit a line over $n = 50 \dots 500$; the per-signer slope ratio is the measured quantity
- Exclude runs with fit $r^2 < 0.99$ from the median: the medians are unchanged without the exclusion

MACHINE	SOFTWARE
AMD Ryzen 5 7520U, powersave, boost on	Python 3.12.3 • libsodium 1.0.18
Intel Xeon @ 2.20 GHz (Kaggle)	Python 3.12.13 • libsodium 1.0.18

P-256 PREDICTED RATIO

bare $T_e/T_a/T_h$ model	1.99
with real hash calls priced	2.37
measured median (range 2.18 – 2.40)	2.37

$n \times \text{Ed25519}$ verification: 47.3 ms vs 177.7 ms aggregate at $n = 500$ (Xeon). Aggregation buys bandwidth, not verifier CPU.

B10 · Testing and reproducibility

AREA	TESTS
Backend algebra	36
Encoding	11
Hashing	27
Scheme	44
Verma	26
Attack and ablation	36
Operation counts	34
Timing harness	7
total	221

101 tests × 2 backends + 19 backend-independent. Self-test:
62 checks.

```
$ make selftest
$ make test
$ make attack
$ .venv/bin/python -m cbas.demo
$ .venv/bin/python -m pytest \
    tests/test_forgery.py -k ablat -vv
$ .venv/bin/python -m pytest \
    tests/test_opcount.py tests/test_verma.py \
    -k "opcount or published_row" -vv
$ .venv/bin/python \
    presentation/audit/verify_claims.py
$ .venv/bin/python \
    presentation/audit/dagger_checks.py
```

Checkpoint: tag presentation-1-interim (881b7b0), branch presentation-1-work.

B11 · Demo fallback: captured output of make attack

```
$ make attack      # Target 1

TARGET 1  Verma et al., CB-CAS (IEEE IoT-J 2020)
honest message      : temp=21.4C;line=A;seq=17
honest sig verifies : True
attacker is the KGC. It holds the master key and the issued
certificate, and has seen exactly one signature.
It does NOT know sk_i.
target message      : temp=-40.0C;line=A;seq=17;SHUTDOWN
forged sig verifies : True    <-- FORGERY SUCCEEDED
forgery survives aggregation : True
```

Demo 1

forged signature verifies: True
it also survives aggregation

```
$ make attack      # Target 2

TARGET 2  Qiao et al., repaired CBAS (IEEE Systems Journal 2023)
honest sig verifies : True
target message      : temp=-40.0C;line=A;seq=17;SHUTDOWN
same attacker, same capabilities, same attack.
forgery succeeded    : False
iterations tried     : 16
candidate T' per iteration (first 8 bytes):
  1. be561edb80eaaad19...
  2. 12e56d96ebfece2f...
  3. 6056990e7f214dfb...
  4. 6c4c65bec8598c35...
  5. 707766e2b93283c7...
  6. d0f9eb291484e581...
  ... 10 more, all distinct
why it fails:
T' must be fixed before  $v' = H1(\dots||T'||\dots)$  can be computed, but
the attack defines  $T' = (v'/v_i)*T_i$  in terms of  $v'$ . The iteration
never settles: forging reduces to finding a fixed point of a random
oracle.
```

Actual captured stdout, 2026-10-07, branch presentation-1-work. Candidate bytes differ on every run.

B12 • Demo fallback: captured output of the ablation tests

```
$ .venv/bin/python -m pytest tests/test_forgery.py -k ablat -vv

collecting ... collected 36 items / 26 deselected / 10 selected
tests/test_forgery.py::test_ablating_fix2_restores_the_forgery[p256-temp=-40.0C;line=A;seq=17;SHUTDOWN] PASSED [ 10%]
tests/test_forgery.py::test_ablating_fix2_restores_the_forgery[p256-pressure=0;valve=OPEN] PASSED [ 20%]
tests/test_forgery.py::test_ablating_fix2_restores_the_forgery[p256-] PASSED [ 30%]
tests/test_forgery.py::test_ablating_fix2_restores_the_forgery[p256-\x00\xff\x ... \xff\x00\xff\x00\xff] PASSED [ 40%]
tests/test_forgery.py::test_ablating_fix2_restores_the_forgery[ristretto255-te ... e=A;seq=17;SHUTDOWN] PASSED [ 50%]
tests/test_forgery.py::test_ablating_fix2_restores_the_forgery[ristretto255-pressure=0;valve=OPEN] PASSED [ 60%]
tests/test_forgery.py::test_ablating_fix2_restores_the_forgery[ristretto255-] PASSED [ 70%]
tests/test_forgery.py::test_ablating_fix2_restores_the_forgery[ristretto255-\x ... \xff\x00\xff\x00\xff] PASSED [ 80%]
tests/test_forgery.py::test_ablated_variant_still_has_fixes_1_and_3[p256] PASSED [ 90%]
tests/test_forgery.py::test_ablated_variant_still_has_fixes_1_and_3[ristretto255] PASSED [100%]
===== 10 passed, 26 deselected in 0.37s =====
```

8 forgery-restored cases (4 messages × 2 backends) + 2 checks that the ablated variant is still a working scheme

Actual captured stdout, 2026-10-07. Long test ids are shortened with an ellipsis.